

# 在 GKE 使用 Airflow 2 建構 MLOps 工作流程

來源：<https://codelabs.developers.google.com/codelabs/cloud-mlops-airflow-gke?hl=zh-tw>

步驟數：20

在本教學課程中，您將瞭解如何透過 Airflow DAG 在 GKE 上使用 vLLM 訓練及執行模型。

---

# 目錄

1. 1. 總覽
2. 2. 平台工程可協助機器學習工程師/科學家
3. 3. Google Cloud 設定和需求
4. 4. 步驟 1：在 Kaggle 上註冊並驗證
5. 5. 步驟 2：在 HuggingFace 註冊並驗證
6. 6. 步驟 3：建立必要的 Google Cloud 基礎架構資源
7. 7. 步驟 4：透過 Helm 資訊套件在 GKE 上安裝 Airflow
8. 8. 步驟 5：使用連線和變數初始化 Airflow
9. 9. 應用程式程式碼容器 #1 - 資料下載
10. 10. 應用程式程式碼容器 #2 - 資料準備
11. 11. 應用程式程式碼容器 #3 - 微調
12. 12. Airflow 2 總覽 (包括什麼是 DAG)
13. 13. DAG 總覽
14. 14. 微調模型與使用 RAG 的比較
15. 15. DAG 工作 #1 - 在 Airflow 上建立第一個步驟：下載資料
16. 16. DAG 工作 #2 - 在 Airflow 上建立第二個步驟：資料準備
17. 17. DAG 工作 #3 - 在 Airflow 上建立第三個步驟：微調模型
18. 18. DAG 工作 #4 - 在 Airflow 上建立最後一個步驟：推論 / 提供模型
19. 19. 恭喜！
20. 20. 在正式環境中執行這項操作

## 1. 總覽



本程式碼研究室將示範如何整合 DevOps 做法與機器學習 (MLOps)，方法是下載資料集、調整模型，然後使用 Airflow DAG 在 Google Kubernetes Engine (GKE) 上部署 LLM，並盡量減少抽象化。因此，我們使用 gcloud 指令，而非 Terraform，方便您逐步完成實驗室，並從平台工程師和機器學習工程師的角度，輕鬆瞭解每個程序。

這份實用指南將逐步說明如何運用 Airflow 簡化 AI 工作流程，並透過設定 DAG，清楚實用展示整個 MLOps 生命週期。

### 課程內容

- 打破知識隔閡並改善工作流程，促進平台和機器學習工程師之間的合作與瞭解
- 瞭解如何在 GKE 上部署、使用及管理 Airflow 2
- 從頭到尾設定 Airflow DAG
- 使用 GKE 建構實際工作環境等級的機器學習系統基礎
- 運用機器學習系統並進行作業
- 瞭解平台工程如何成為 MLOps 的重要支援支柱

### 本程式碼研究室的目標

- 您可以向 LLM 詢問電影相關問題，這類 LLM 是以 Gemma-2-9b-it 為基礎微調，並透過 vLLM 在 GKE 中提供服務。

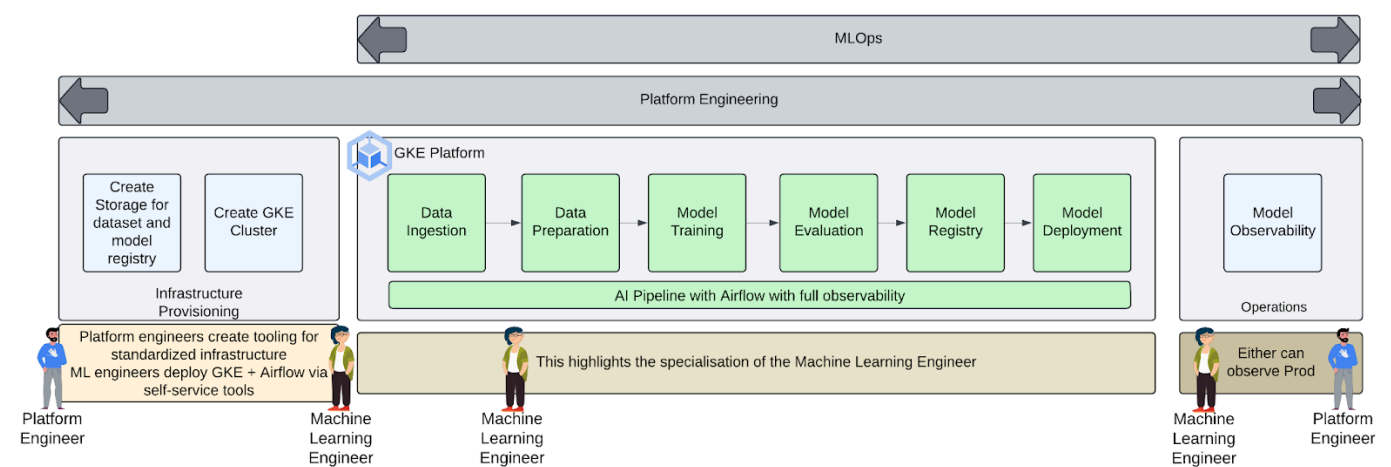
### 目標對象

- 機器學習工程師
- 平台工程師
- 數據資料學家
- 資料工程師
- DevOps 工程師
- Platform Architect
- 客戶工程師

### 本程式碼研究室不適用於

- 做為 GKE 或 AI/機器學習工作流程的簡介
  - 全面瞭解 Airflow 功能集
-

## 2. 平台工程可協助機器學習工程師/科學家



平台工程和 MLOps 是相互依存的領域，兩者會協同合作，為機器學習開發和部署作業建立穩健有效率的環境。

**範圍：**平台工程的範圍比 MLOps 更廣，涵蓋整個軟體開發生命週期，並為其提供工具和基礎架構。

MLOps 填補了機器學習開發、部署和推論之間的鴻溝。

**專業知識：**平台工程師通常在基礎架構技術方面具備豐富的專業知識，例如雲端運算、容器化和資料管理。

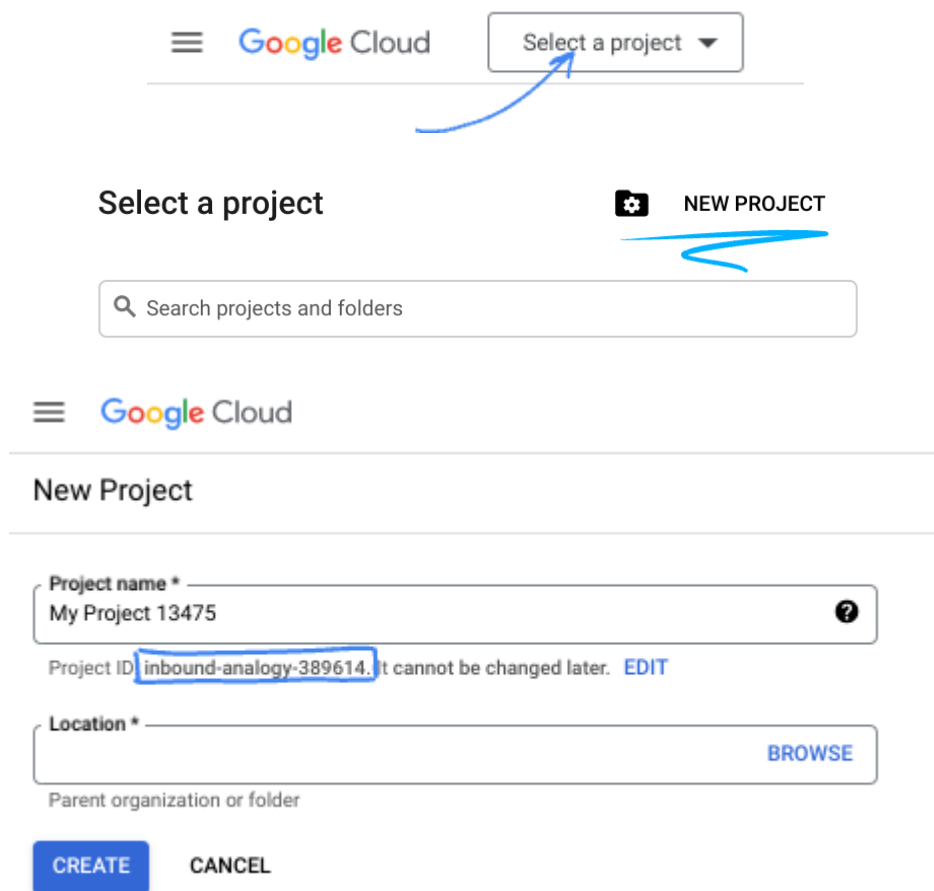
MLOps 工程師專精於機器學習模型開發、部署和監控，通常具備資料科學和軟體工程技能。

**工具：**平台工程師會建立基礎架構佈建、設定管理、容器自動調度管理和應用程式架構的工具。機器學習運作工程師會使用工具訓練、實驗、部署、監控及控管機器學習模型版本。

## 3. Google Cloud 設定和需求

### 自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。



The screenshot displays the Google Cloud console interface. At the top, there's a navigation bar with the Google Cloud logo and a 'Select a project' dropdown menu. Below this, the 'Select a project' page is shown, featuring a search bar labeled 'Search projects and folders' and a 'NEW PROJECT' button. The 'New Project' form is visible below, with fields for 'Project name \*', 'Project ID', 'Location \*', and 'Parent organization or folder'. The 'Project ID' field is highlighted with a blue box, showing the value 'inbound-analogy-389614'. The 'Location' field has a 'BROWSE' button next to it. At the bottom of the form are 'CREATE' and 'CANCEL' buttons.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

**注意：**專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

**注意：**如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

## 啟動 Cloud Shell

雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

### 啟用 Cloud Shell

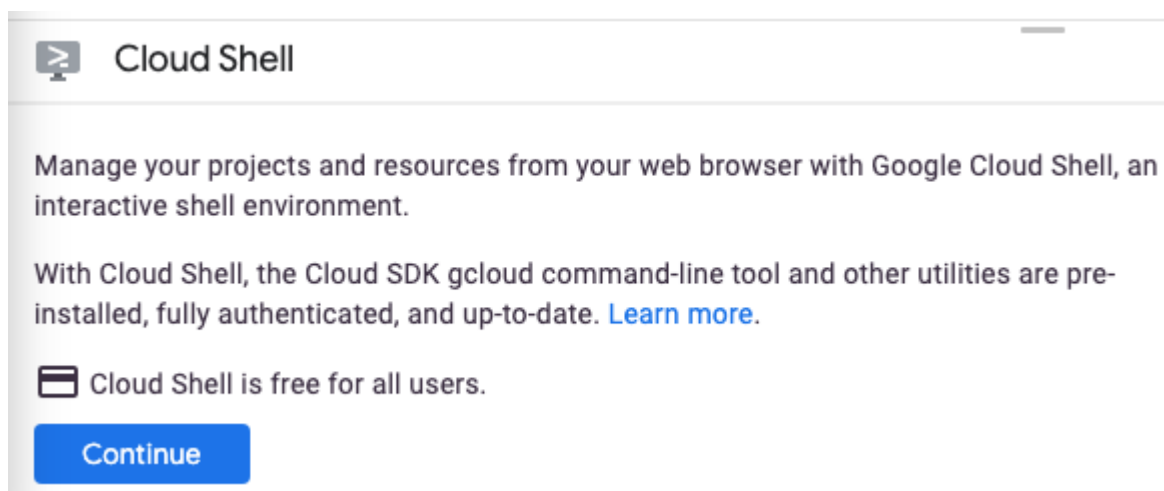
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



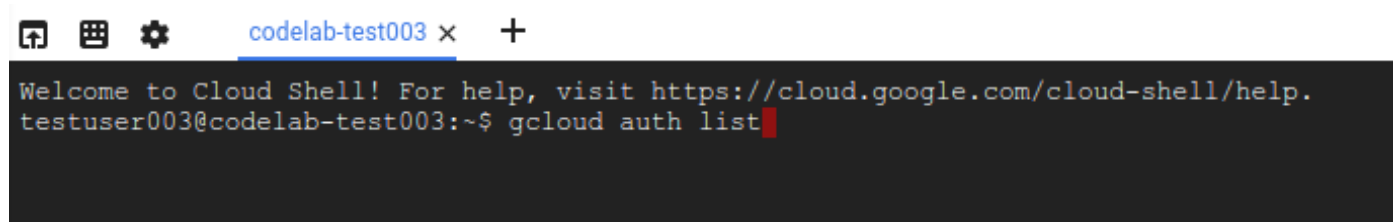
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

#### Credentialed Accounts

ACTIVE ACCOUNT

\* <my\_account>@<my\_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

**注意：** `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

#### 指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

#### 指令輸出

```
Updated property [core/project].
```

**注意：** 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

## 4. 步驟 1：在 Kaggle 上註冊並驗證

如要開始 CodeLab，您必須在 Kaggle 上建立帳戶。Kaggle 是 Google 擁有的線上社群平台，專為資料科學家和機器學習愛好者而設，並提供各種領域的大量公開資料集。您將從這個網站下載 RottenTomatoes 資料集，用於訓練模型。

- 註冊 [Kaggle](#)，您可以使用 Google SSO 登入
  - 接受條款及細則
  - 前往「設定」並取得使用者名稱 [使用者名稱](#)
  - 在「API」部分下方，選取「Create new token from」Kaggle，系統會下載 kaggle.json
  - 如有任何問題，請前往[這個](#)支援頁面
-

步驟 5 · 約 0 分鐘

## 5. 步驟 2：在 HuggingFace 註冊並驗證

HuggingFace 是集中式平台，可供任何人使用機器學習技術。這個平台代管 90 萬個模型、20 萬個資料集和 30 萬個示範應用程式 (Spaces)，全部都是開放原始碼且公開提供。

- 註冊 [HuggingFace](#) - 建立帳戶並設定使用者名稱，無法使用 Google 單一登入
  - 確認您的電子郵件地址
  - 前往[這裡](#)，接受 Gemma-2-9b-it 模型授權
  - [在這裡](#)建立 HuggingFace 權杖
  - 記下權杖憑證，稍後會用到
-

## 6. 步驟 3：建立必要的 Google Cloud 基礎架構資源

您將設定 GKE、GCE、Artifact Registry，並使用[工作負載身分聯盟](#)套用 IAM 角色。

您的 AI 工作流程會使用兩個節點集區，一個用於訓練，另一個用於推論。訓練節點集使用配備一個 Nvidia L4 Tensor Core GPU 的 g2-standard-8 GCE VM。推論節點集使用配備兩個 Nvidia L4 Tensor Core GPU 的 g2-standard-24 VM。指定地區時，請選擇支援所需 GPU 的地區 ([連結](#))。

在 Cloud Shell 中執行下列指令：

```
# Set environment variables
export CODELAB_PREFIX=mlops-airflow
export PROJECT_NUMBER=$(gcloud projects list --filter="${DEVSHHELL_PROJECT_ID}" --
format="value(PROJECT_NUMBER)")

SUFFIX=$(echo $RANDOM | md5sum | head -c 4; echo;)
export CLUSTER_NAME=${CODELAB_PREFIX}
export CLUSTER_SA=sa-${CODELAB_PREFIX}
export BUCKET_LOGS_NAME=${CODELAB_PREFIX}-logs-${SUFFIX}
export BUCKET_DAGS_NAME=${CODELAB_PREFIX}-dags-${SUFFIX}
export BUCKET_DATA_NAME=${CODELAB_PREFIX}-data-${SUFFIX}
export REPO_NAME=${CODELAB_PREFIX}-repo
export REGION=us-central1

# Enable Google API's
export PROJECT_ID=${DEVSHHELL_PROJECT_ID}
gcloud config set project ${PROJECT_ID}
gcloud services enable \
container.googleapis.com \
cloudbuild.googleapis.com \
artifactregistry.googleapis.com \
storage.googleapis.com
```

```

# Create a VPC for the GKE cluster
gcloud compute networks create mlops --subnet-mode=auto

# Create IAM and the needed infrastructure (GKE, Bucket, Artifact Registry)
# Create an IAM Service Account
gcloud iam service-accounts create ${CLUSTER_SA} --display-name="SA for ${CLUSTER_NAME}"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"serviceAccount:${CLUSTER_SA}@${DEVSHHELL_PROJECT_ID}.iam.gserviceaccount.com" --role
roles/container.defaultNodeServiceAccount

# Create a GKE cluster
gcloud container clusters create ${CLUSTER_NAME} --zone ${REGION}-a --num-nodes=4 --
network=mlops --create-subnetwork name=mlops-subnet --enable-ip-alias --addons
GcsFuseCsiDriver --workload-pool=${DEVSHHELL_PROJECT_ID}.svc.id.goog --no-enable-insecure-
kubelet-readonly-port --service-
account=${CLUSTER_SA}@${DEVSHHELL_PROJECT_ID}.iam.gserviceaccount.com

# Create 1 x node pool for our cluster 1 x node with 1 x L4 GPU for model finetuning
gcloud container node-pools create training \
  --accelerator type=nvidia-l4,count=1,gpu-driver-version=latest \
  --project=${PROJECT_ID} \
  --location=${REGION}-a \
  --node-locations=${REGION}-a \
  --cluster=${CLUSTER_NAME} \
  --machine-type=g2-standard-12 \
  --num-nodes=1

# Create 1 x node pool for our cluster 1 x node with 2 x L4 GPUs for inference
gcloud container node-pools create inference\
  --accelerator type=nvidia-l4,count=2,gpu-driver-version=latest \
  --project=${PROJECT_ID} \
  --location=${REGION}-a \
  --node-locations=${REGION}-a \
  --cluster=${CLUSTER_NAME} \
  --machine-type=g2-standard-24 \
  --num-nodes=1

# Download K8s credentials
gcloud container clusters get-credentials ${CLUSTER_NAME} --location ${REGION}-a

# Create Artifact Registry
gcloud artifacts repositories create ${REPO_NAME} --repository-format=docker --
location=${REGION}
gcloud artifacts repositories add-iam-policy-binding ${REPO_NAME} --
member=serviceAccount:${CLUSTER_SA}@${DEVSHHELL_PROJECT_ID}.iam.gserviceaccount.com --
role=roles/artifactregistry.reader --location=${REGION}

```

建立 YAML 資訊清單

```
mkdir manifests
cd manifests
```

#### mlops-sa.yaml

```
apiVersion: v1
kind: ServiceAccount
automountServiceAccountToken: true
metadata:
  name: airflow-mlops-sa
  namespace: airflow
  labels:
    tier: airflow
```

#### pv-dags.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: airflow-dags
spec:
  accessModes:
    - ReadWriteMany
  capacity:
    storage: 5Gi
  storageClassName: standard
  mountOptions:
    - implicit-dirs
  csi:
    driver: gcsfuse.csi.storage.gke.io
    volumeHandle: BUCKET_DAGS_NAME
    volumeAttributes:
      gcsfuseLoggingSeverity: warning
```

#### pv-logs.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: airflow-logs
spec:
  accessModes:
    - ReadWriteMany
  capacity:
    storage: 100Gi
  storageClassName: standard
  mountOptions:
    - implicit-dirs
  csi:
    driver: gcsfuse.csi.storage.gke.io
    volumeHandle: BUCKET_LOGS_NAME
    volumeAttributes:
      gcsfuseLoggingSeverity: warning
```

#### pvc-dags.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: airflow-dags
  namespace: airflow
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
  volumeName: airflow-dags
  storageClassName: standard
```

#### pvc-logs.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: airflow-logs
  namespace: airflow
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 100Gi
  volumeName: airflow-logs
  storageClassName: standard
```

## namespace.yaml

```
kind: Namespace
apiVersion: v1
metadata:
  name: airflow
  labels:
    name: airflow
```

## sa-role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: airflow
  name: airflow-deployment-role
rules:
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["create", "get", "list", "watch", "update", "patch", "delete"]
- apiGroups: [""]
  resources: ["services"]
  verbs: ["create", "get", "list", "watch", "patch", "update", "delete"]
```

## sa-rolebinding.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: airflow-deployment-rolebinding
  namespace: airflow
subjects:
- kind: ServiceAccount
  name: airflow-worker
  namespace: airflow
roleRef:
  kind: Role
  name: airflow-deployment-role
  apiGroup: rbac.authorization.k8s.io
```

## inference.yaml

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: inference-deployment
  namespace: airflow
spec:
  replicas: 1
  selector:
    matchLabels:
      app: gemma-server
  template:
    metadata:
      labels:
        app: gemma-server
        ai.gke.io/model: gemma-2-9b-it
        ai.gke.io/inference-server: vllm
      annotations:
        gke-gcsfuse/volumes: "true"
    spec:
      serviceAccountName: airflow-mlops-sa
      tolerations:
        - key: "nvidia.com/gpu"
          operator: "Exists"
          effect: "NoSchedule"
        - key: "on-demand"
          value: "true"
          operator: "Equal"
          effect: "NoSchedule"
      containers:
        - name: inference-server
          image: vllm/vllm-openai:v0.6.6
          ports:
            - containerPort: 8000
          resources:
            requests:
              nvidia.com/gpu: "2"
            limits:
              nvidia.com/gpu: "2"
          command: ["/bin/sh", "-c"]
          args:
            - |
              python3 -m vllm.entrypoints.api_server --model=/modeldata/fine_tuned_model --
tokenizer=/modeldata/fine_tuned_model --tensor-parallel-size=2
          volumeMounts:
            - mountPath: /dev/shm
              name: dshm
            - name: gcs-fuse-csi-ephemeral
              mountPath: /modeldata
              readOnly: true
      volumes:
        - name: dshm
          emptyDir:

```

```

    medium: Memory
  - name: gcs-fuse-csi-ephemeral
    csi:
      driver: gcsfuse.csi.storage.gke.io
      volumeAttributes:
        bucketName: BUCKET_DATA_NAME
        mountOptions: "implicit-dirs,file-cache:enable-parallel-downloads:true,file-
cache:max-parallel-downloads:-1"
        fileCacheCapacity: "20Gi"
        fileCacheForRangeRead: "true"
        metadataStatCacheCapacity: "-1"
        metadataTypeCacheCapacity: "-1"
        metadataCacheTTLSeconds: "-1"
    nodeSelector:
      cloud.google.com/gke-accelerator: nvidia-l4

```

inference-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: llm-service
  namespace: airflow
spec:
  selector:
    app: gemma-server
  type: LoadBalancer
  ports:
    - protocol: TCP
      port: 8000
      targetPort: 8000

```

建立 3 個 Google Cloud Storage (GCS) bucket

```
gcloud storage buckets create gs://${BUCKET_LOGS_NAME} --location=${REGION}
gcloud storage buckets create gs://${BUCKET_DAGS_NAME} --location=${REGION}
gcloud storage buckets create gs://${BUCKET_DATA_NAME} --location=${REGION}
```

```
# Create the namespace in GKE
kubectl apply -f namespace.yaml
```

```
# Create the PV and PVC in GKE for Airflow DAGs storage
sed -i "s/BUCKET_DAGS_NAME/${BUCKET_DAGS_NAME}/g" pv-dags.yaml
sed -i "s/BUCKET_LOGS_NAME/${BUCKET_LOGS_NAME}/g" pv-logs.yaml
sed -i "s/BUCKET_DATA_NAME/${BUCKET_DATA_NAME}/g" inference.yaml
kubectl apply -f pv-dags.yaml
kubectl apply -f pv-logs.yaml
kubectl apply -f pvc-dags.yaml
kubectl apply -f pvc-logs.yaml
kubectl apply -f mlops-sa.yaml
kubectl apply -f sa-role.yaml
kubectl apply -f sa-rolebinding.yaml
```

Add the necessary IAM roles to access buckets from Airflow using Workload Identity Federation

```
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-scheduler" --role
"roles/storage.objectUser"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-triggerer" --role
"roles/storage.objectUser"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-worker" --role
"roles/storage.objectUser"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-worker" --role
"roles/container.developer"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-mlops-sa" --role
"roles/artifactregistry.reader"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-webserver" --role
"roles/storage.objectUser"
gcloud projects add-iam-policy-binding ${DEVSHHELL_PROJECT_ID} --member
"principal://iam.googleapis.com/projects/${PROJECT_NUMBER}/locations/global/workloadIdentityP
ools/${DEVSHHELL_PROJECT_ID}.svc.id.goog/subject/ns/airflow/sa/airflow-mlops-sa" --role
"roles/storage.objectUser"
```



步驟 7 · 約 1 分鐘

## 7. 步驟 4：透過 Helm 資訊套件在 GKE 上安裝 Airflow

現在使用 Helm 部署 Airflow 2。Apache Airflow 是開放原始碼的[工作流程管理平台](#)，適用於資料工程管道。我們稍後會深入探討 Airflow 2 的功能集。

Airflow Helm 圖表的 values.yaml

```
config:
  webserver:
    expose_config: true
webserver:
  service:
    type: LoadBalancer
  podAnnotations:
    gke-gcsfuse/volumes: "true"
executor: KubernetesExecutor
extraEnv: |-
  - name: AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL
    value: "30"
logs:
  persistence:
    enabled: true
    existingClaim: "airflow-logs"
dags:
  persistence:
    enabled: true
    existingClaim: "airflow-dags"
scheduler:
  podAnnotations:
    gke-gcsfuse/volumes: "true"
triggerer:
  podAnnotations:
    gke-gcsfuse/volumes: "true"
workers:
  podAnnotations:
    gke-gcsfuse/volumes: "true"
```

部署 Airflow 2

```
helm repo add apache-airflow https://airflow.apache.org
helm repo update

helm upgrade --install airflow apache-airflow/airflow --namespace airflow -f values.yaml
```

## 8. 步驟 5：使用連線和變數初始化 Airflow

部署 Airflow 2 後，即可開始設定。我們定義了一些變數，這些變數會由 Python 指令碼讀取。

1. 使用瀏覽器透過通訊埠 8080 存取 Airflow UI

取得外部 IP

```
kubectl -n airflow get svc/airflow-webserver --output
jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

開啟網路瀏覽器並前往 `http://<EXTERNAL-IP>:8080`。登入資訊為 `admin / admin`

2. 在 Airflow UI 中建立預設 GCP 連線，因此請依序前往「Admin」→「Connections」→「+ Add a new record」

- 連線 ID：google\_cloud\_default
- 連線類型：Google Cloud

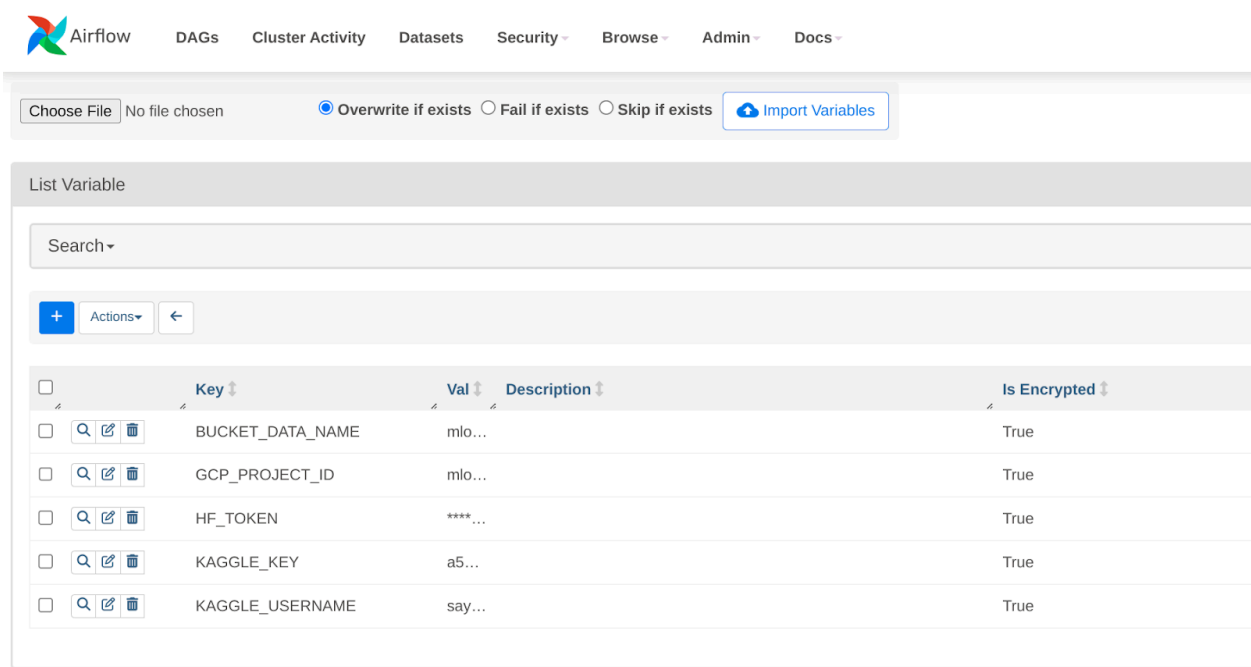
按一下「儲存」。

3. 建立所需變數，因此請依序前往「管理」→「變數」→「+ 新增記錄」

- 鍵：BUCKET\_DATA\_NAME - 值：從 `echo $BUCKET_DATA_NAME` 複製
- 鍵：GCP\_PROJECT\_ID - 值：從 `echo $DEVSHHELL_PROJECT_ID` 複製
- 鍵：HF\_TOKEN - 值：插入 HF 權杖
- 鍵：KAGGLE\_USERNAME - 值：插入您的 Kaggle 使用者名稱
- 鍵：KAGGLE\_KEY - 值：從 `kaggle.json` 複製這個值

輸入每個鍵/值組合後，請按一下「儲存」。

您的 UI 應如下所示：



步驟 9 · 約 0 分鐘

## 9. 應用程式程式碼容器 #1 - 資料下載

在這個 Python 指令碼中，我們會向 Kaggle 進行驗證，將資料集下載至 GCS bucket。

指令碼本身已容器化，因為這會成為 DAG 單位 #1，且我們預期資料集會經常更新，因此希望自動執行這項作業。

在這裡建立目錄並複製我們的指令碼

```
cd .. ; mkdir 1-dataset-download
cd 1-dataset-download
```

dataset-download.py

```
import os
import kagglehub
from google.cloud import storage

KAGGLE_USERNAME = os.getenv("KAGGLE_USERNAME")
KAGGLE_KEY = os.getenv("KAGGLE_KEY")
BUCKET_DATA_NAME = os.getenv("BUCKET_DATA_NAME")

def upload_blob(bucket_name, source_file_name, destination_blob_name):
    """Uploads a file to the bucket."""
    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    blob = bucket.blob(destination_blob_name)
    blob.upload_from_filename(source_file_name)
    print(f"File {source_file_name} uploaded to {destination_blob_name}.")

# Download latest version
path = kagglehub.dataset_download("priyamchoksi/rotten-tomato-movie-reviews-1-44m-rows")

print("Path to dataset files:", path)
destination_blob_name = "rotten_tomatoes_movie_reviews.csv"
source_file_name = f"{path}/{destination_blob_name}"

upload_blob(BUCKET_DATA_NAME, source_file_name, destination_blob_name)
```

Dockerfile

```
FROM python:3.13.0-slim-bookworm
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r requirements.txt
COPY dataset-download.py .
CMD ["python", "dataset-download.py"]
```

requirements.txt

```
google-cloud-storage==2.19.0  
kagglehub==0.3.4
```

現在，我們要為 dataset-download 建立容器映像檔，並推送至 Artifact Registry

```
gcloud builds submit --tag ${REGION}-  
docker.pkg.dev/${DEVSHIELD_PROJECT_ID}/${REPO_NAME}/dataset-download:latest
```

---

步驟 10 · 約 0 分鐘

## 10. 應用程式程式碼容器 #2 - 資料準備

在資料準備步驟中，我們達成以下目標：

1. 指定要用於微調基礎模型的資料集大小
2. 載入資料集，也就是將 CSV 檔案讀取至 Pandas DataFrame，這是資料列和資料欄的二維資料結構
3. 資料轉換 / 前處理 - 指定要保留的資料，藉此判斷資料集中哪些部分不相關，實際上就是移除其餘部分
4. 將 `transform` 函式套用至 DataFrame 的每個資料列
5. 將準備好的資料儲存回 GCS 值區

在這裡建立目錄並複製我們的指令碼

```
cd .. ; mkdir 2-data-preparation  
cd 2-data-preparation
```

data-preparation.py

```

import os
import pandas as pd
import gcsfs
import json
from datasets import Dataset

# Environment variables
GCP_PROJECT_ID = os.getenv("GCP_PROJECT_ID")
BUCKET_DATA_NAME = os.getenv("BUCKET_DATA_NAME")

DATASET_NAME = os.getenv("DATASET_NAME", "rotten_tomatoes_movie_reviews.csv")
PREPARED_DATASET_NAME = os.getenv("PREPARED_DATA_URL", "prepared_data.jsonl")
DATASET_LIMIT = int(os.getenv("DATASET_LIMIT", "100")) # Process a limited number of rows,
used 100 during testing phase but can be increased

DATASET_URL = f"gs://{BUCKET_DATA_NAME}/{DATASET_NAME}"
PREPARED_DATASET_URL = f"gs://{BUCKET_DATA_NAME}/{PREPARED_DATASET_NAME}"

# Load the dataset
print(f"Loading dataset from {DATASET_URL}...")

def transform(data):
    """
    Transforms a row of the DataFrame into the desired format for fine-tuning.

    Args:
        data: A pandas Series representing a row of the DataFrame.

    Returns:
        A dictionary containing the formatted text.
    """
    question = f"Review analysis for movie '{data['id']}'"
    context = data['reviewText']
    answer = data['scoreSentiment']
    template = "Question: {question}\nContext: {context}\nAnswer: {answer}"
    return {'text': template.format(question=question, context=context, answer=answer)}

try:
    df = pd.read_csv(DATASET_URL, nrows=DATASET_LIMIT)
    print(f"Dataset loaded successfully.")

    # Drop rows with NaN values in relevant columns
    df = df.dropna(subset=['id', 'reviewText', 'scoreSentiment'])

    # Apply transformation to the DataFrame
    transformed_data = df.apply(transform, axis=1).tolist()

    # Convert transformed data to a DataFrame and then to a Hugging Face Dataset
    transformed_df = pd.DataFrame(transformed_data)
    dataset = Dataset.from_pandas(transformed_df)

    # Save the prepared dataset to JSON lines format

```

```
with gcsfs.GCSFileSystem(project=GCP_PROJECT_ID).open(PREPARED_DATASET_URL, 'w') as f:
    for item in dataset:
        f.write(json.dumps(item) + "\n")

print(f"Prepared dataset saved to {PREPARED_DATASET_URL}")

except Exception as e:
    print(f"Error during data loading or preprocessing: {e}")
    import traceback
    print(traceback.format_exc())
```

## Dockerfile

```
FROM python:3.13.0-slim-bookworm
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r
requirements.txt
COPY data-preparation.py .
CMD ["python", "data-preparation.py"]
```

## requirements.txt

```
datasets==3.1.0
gcsfs==2024.9.0
pandas==2.2.3
```

# Now we create a container images for data-preparation and push it to the Artifact Registry

```
gcloud builds submit --tag ${REGION}-docker.pkg.dev/${DEVSHIELD_PROJECT_ID}/${REPO_NAME}/data-
preparation:latest
```

---

## 11. 應用程式程式碼容器 #3 - 微調

我們以 Gemma-2-9b-it 做為基礎模型，然後使用新的資料集微調模型。

這些是微調步驟期間發生的步驟順序。

1. **設定**：匯入程式庫、定義參數 (適用於模型、資料和訓練)，並從 Google Cloud Storage 載入資料集。
2. **載入模型**：載入預先訓練的語言模型，並進行量化以提高效率，同時載入對應的權杖化工具。
3. **設定 LoRA**：設定低秩調整 (LoRA)，加入可訓練的小型矩陣，有效微調模型。
4. **訓練**：定義訓練參數，並使用 `SFTTrainer`，以 `FP16` 量化類型，在載入的資料集上微調模型。
5. **儲存及上傳**：將微調模型和權杖化工具儲存至本機，然後上傳至我們的 GCS bucket。

接著，我們使用 Cloud Build 建立容器映像檔，並儲存在 Artifact Registry 中。

在這裡建立目錄並複製我們的指令碼

```
cd .. ; mkdir 3-fine-tuning
cd 3-fine-tuning
```

finetuning.py

```

import os
import torch
import bitsandbytes
from accelerate import Accelerator
from datasets import Dataset, load_dataset, load_from_disk
from peft import LoraConfig, PeftModel
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import DataCollatorForCompletionOnlyLM, SFTConfig, SFTTrainer
from google.cloud import storage

# Environment variables
BUCKET_DATA_NAME = os.environ["BUCKET_DATA_NAME"]
PREPARED_DATA_URL = os.getenv("PREPARED_DATA_URL", "prepared_data.jsonl")
# Finetuned model name
new_model = os.getenv("NEW_MODEL_NAME", "fine_tuned_model")
# Base model from the Hugging Face hub
model_name = os.getenv("MODEL_ID", "google/gemma-2-9b-it")
# Root path for saving the finetuned model
save_model_path = os.getenv("MODEL_PATH", "./output")

# Load tokenizer
print("Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right" # Fix weird overflow issue with fp16 training
print("Tokenizer loaded successfully!")

# Load dataset
EOS_TOKEN = tokenizer.eos_token
dataset = load_dataset(
    "json", data_files=f"gs://{BUCKET_DATA_NAME}/{PREPARED_DATA_URL}", split="train")
print(dataset)

#####
# LoRA parameters
#####
# LoRA attention dimension
lora_r = int(os.getenv("LORA_R", "8"))
# Alpha parameter for LoRA scaling
lora_alpha = int(os.getenv("LORA_ALPHA", "16"))
# Dropout probability for LoRA layers
lora_dropout = float(os.getenv("LORA_DROPOUT", "0.1"))

#####
# TrainingArguments parameters
#####
# Number of training epochs
num_train_epochs = int(os.getenv("EPOCHS", 1))
# Set fp16/bf16 training (set bf16 to True with an A100)
fp16 = False
bf16 = False
# Batch size per GPU for training

```

```

per_device_train_batch_size = int(os.getenv("TRAIN_BATCH_SIZE", "1"))
# Batch size per GPU for evaluation
per_device_eval_batch_size = 1
# Number of update steps to accumulate the gradients for
gradient_accumulation_steps = int(os.getenv("GRADIENT_ACCUMULATION_STEPS", "1"))
# Enable gradient checkpointing
gradient_checkpointing = True
# Maximum gradient normal (gradient clipping)
max_grad_norm = 0.3
# Initial learning rate (AdamW optimizer)
learning_rate = 2e-4
# Weight decay to apply to all layers except bias/LayerNorm weights
weight_decay = 0.001
# Optimizer to use
optim = "paged_adamw_32bit"
# Learning rate schedule
lr_scheduler_type = "cosine"
# Number of training steps (overrides num_train_epochs)
max_steps = -1
# Ratio of steps for a linear warmup (from 0 to learning rate)
warmup_ratio = 0.03

# Group sequences into batches with same length
# Saves memory and speeds up training considerably
group_by_length = True
# Save strategy: steps, epoch, no
save_strategy = os.getenv("CHECKPOINT_SAVE_STRATEGY", "steps")
# Save total limit of checkpoints
save_total_limit = int(os.getenv("CHECKPOINT_SAVE_TOTAL_LIMIT", "5"))
# Save checkpoint every X updates steps
save_steps = int(os.getenv("CHECKPOINT_SAVE_STEPS", "1000"))
# Log every X updates steps
logging_steps = 50

#####
# SFT parameters
#####
# Maximum sequence length to use
max_seq_length = int(os.getenv("MAX_SEQ_LENGTH", "512"))
# Pack multiple short examples in the same input sequence to increase efficiency
packing = False

# Load base model
print(f"Loading base model started")
model = AutoModelForCausalLM.from_pretrained(
    attn_implementation="eager",
    pretrained_model_name_or_path=model_name,
    torch_dtype=torch.float16,
)
model.config.use_cache = False
model.config.pretraining_tp = 1
print("Loading base model completed")

```

```

# Configure fine-tuning with LoRA
print(f"Configuring fine tuning started")
peft_config = LoraConfig(
    lora_alpha=lora_alpha,
    lora_dropout=lora_dropout,
    r=lora_r,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
)

# Set training parameters
training_arguments = SFTConfig(
    bf16=bf16,
    dataset_kwargs={
        "add_special_tokens": False,
        "append_concat_token": False,
    },
    dataset_text_field="text",
    disable_tqdm=True,
    fp16=fp16,
    gradient_accumulation_steps=gradient_accumulation_steps,
    gradient_checkpointing=gradient_checkpointing,
    gradient_checkpointing_kwargs={"use_reentrant": False},
    group_by_length=group_by_length,
    log_on_each_node=False,
    logging_steps=logging_steps,
    learning_rate=learning_rate,
    lr_scheduler_type=lr_scheduler_type,
    max_grad_norm=max_grad_norm,
    max_seq_length=max_seq_length,
    max_steps=max_steps,
    num_train_epochs=num_train_epochs,
    optim=optim,
    output_dir=save_model_path,
    packing=packing,
    per_device_train_batch_size=per_device_train_batch_size,
    save_strategy=save_strategy,
    save_steps=save_steps,
    save_total_limit=save_total_limit,
    warmup_ratio=warmup_ratio,
    weight_decay=weight_decay,
)

```

```

print(f"Configuring fine tuning completed")

# Initialize the SFTTrainer
print(f"Creating trainer started")
trainer = SFTTrainer(
    model=model,
    train_dataset=dataset,
    peft_config=peft_config,
    dataset_text_field="text",
    max_seq_length=max_seq_length,
    tokenizer=tokenizer,
    args=training_arguments,
    packing=packing,
)

print(f"Creating trainer completed")

# Finetune the model
print("Starting fine-tuning...")
trainer.train()
print("Fine-tuning completed.")

# Save the fine-tuned model
print("Saving new model started")
trainer.model.save_pretrained(new_model)
print("Saving new model completed")

# Merge LoRA weights with the base model
print(f"Merging the new model with base model started")
base_model = AutoModelForCausalLM.from_pretrained(
    low_cpu_mem_usage=True,
    pretrained_model_name_or_path=model_name,
    return_dict=True,
    torch_dtype=torch.float16,
)

model = PeftModel.from_pretrained(
    model=base_model,
    model_id=new_model,
)
model = model.merge_and_unload()

print(f"Merging the new model with base model completed")

accelerator = Accelerator()
print(f"Accelerate unwrap model started")
unwrapped_model = accelerator.unwrap_model(model)
print(f"Accelerate unwrap model completed")

print(f"Save unwrapped model started")
unwrapped_model.save_pretrained(

```

```

        is_main_process=accelerator.is_main_process,
        save_directory=save_model_path,
        save_function=accelerator.save,
    )
print(f"Save unwrapped model completed")

print(f"Save new tokenizer started")
if accelerator.is_main_process:
    tokenizer.save_pretrained(save_model_path)
print(f"Save new tokenizer completed")

# Upload the model to GCS
def upload_to_gcs(bucket_name, model_dir):
    """Uploads a directory to GCS."""
    storage_client = storage.Client()
    bucket = storage_client.bucket(bucket_name)
    for root, _, files in os.walk(model_dir):
        for file in files:
            local_file_path = os.path.join(root, file)
            gcs_file_path = os.path.relpath(local_file_path, model_dir)
            blob = bucket.blob(os.path.join(new_model, gcs_file_path)) # Use new_model_name
            blob.upload_from_filename(local_file_path)

# Upload the fine-tuned model and tokenizer to GCS
upload_to_gcs(BUCKET_DATA_NAME, save_model_path)
print(f"Fine-tuned model {new_model} successfully uploaded to GCS.")

```

Dockerfile

```
# Using the NVIDIA CUDA base image
FROM nvidia/cuda:12.6.2-runtime-ubuntu22.04

# Install necessary system packages
RUN apt-get update && \
    apt-get -y --no-install-recommends install python3-dev gcc python3-pip git && \
    rm -rf /var/lib/apt/lists/*

# Copy requirements.txt into the container
COPY requirements.txt .

# Install Python packages from requirements.txt
RUN pip install --no-cache-dir --upgrade pip && pip install --no-cache-dir -r
requirements.txt

# Copy your finetune script into the container
COPY finetuning.py .

# Set the environment variable to ensure output is flushed
ENV PYTHONUNBUFFERED 1
ENV MODEL_ID "google/gemma-2-9b-it"
ENV GCS_BUCKET "finetuning-data-bucket"

# Set the command to run the finetuning script with CUDA device
CMD ["python3", "finetuning.py"]
```

requirements.txt

```
accelerate==1.1.1
bitsandbytes==0.45.0
datasets==3.1.0
gcsfs==2024.9.0
peft==v0.13.2
torch==2.5.1
transformers==4.47.0
trl==v0.11.4
```

現在我們要建立微調用的容器映像檔，並推送至 Artifact Registry

```
gcloud builds submit --tag ${REGION}-
docker.pkg.dev/${DEVSHIELD_PROJECT_ID}/${REPO_NAME}/finetuning:latest
```

## 12. Airflow 2 總覽 (包括什麼是 DAG)

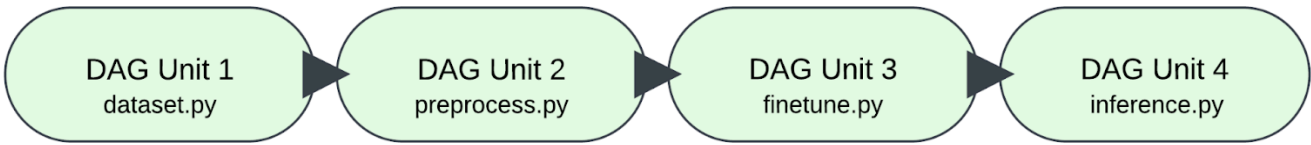
Airflow 是用於自動化調度管理工作流程和資料管道的平台，並使用 DAG (有向無環圖) 在 Python 程式碼中定義這些工作流程，以視覺化方式呈現工作及其依附元件。

Airflow 具有靜態 DAG 和以 Python 為基礎的定義，非常適合用於排定及管理預先定義的工作流程。其架構包含易於使用的 UI，可監控及管理這些工作流程。

基本上，您可以使用 Python 定義、排定及監控資料管道，因此 Airflow 是自動化調度管理工作流程的彈性強大工具。

---

## 13. DAG 總覽



DAG 是指有向非循環圖，在 Airflow 中，DAG 本身代表整個工作流程或管道。定義工作、依附元件和執行順序。

DAG 內的工作流程單元會從 GKE 叢集上的 Pod 執行，並從 Airflow 設定啟動。

摘要：

**Airflow：資料下載** - 這項指令碼會自動從 Kaggle 取得電影評論資料集，並儲存在 GCS bucket 中，方便您在雲端環境中進一步處理或分析。

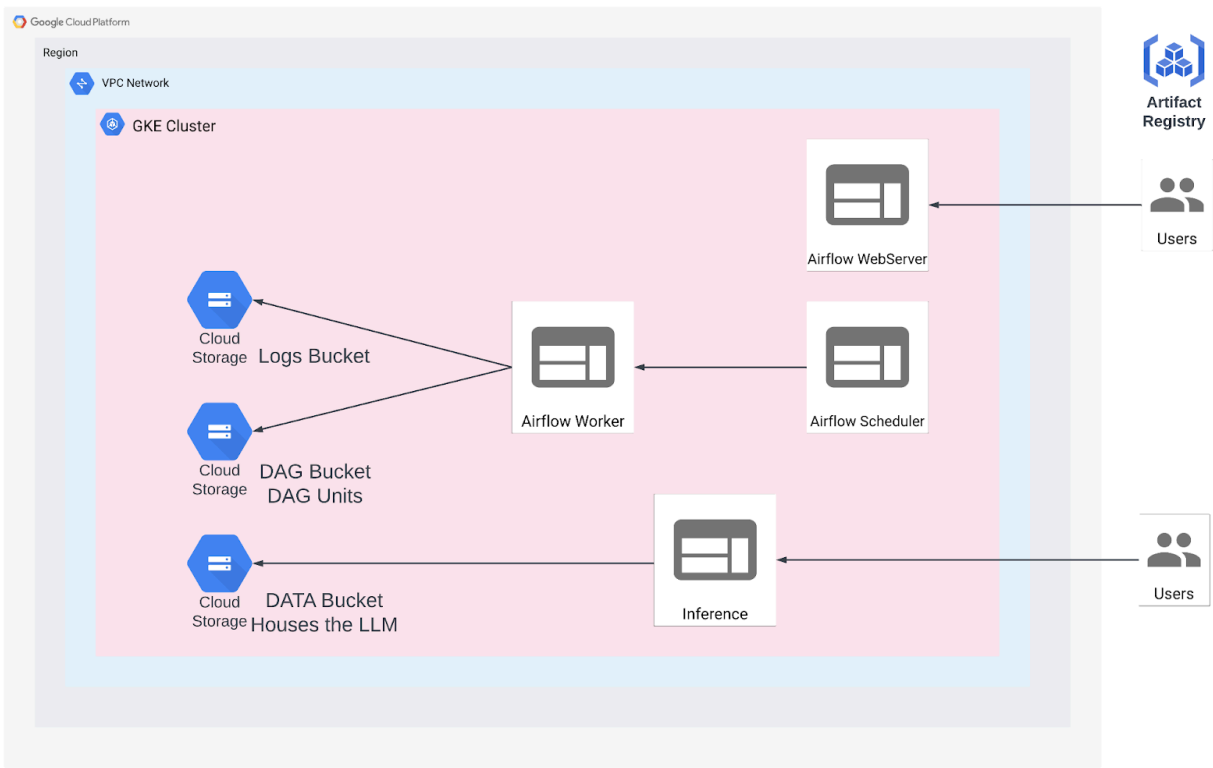
**Airflow：資料準備** - 這段程式碼會取得原始電影評論資料集、移除我們應用情境不需要的額外資料欄，並刪除缺少值的資料集。接著，將資料集建構成適合機器學習的問答格式，並儲存回 GCS 以供日後使用。

**Airflow：模型微調** - 這段程式碼會使用 LoRA (低秩適應) 技術微調大型語言模型 (LLM)，然後儲存更新後的模型。首先，從 Google Cloud Storage 載入預先訓練的 LLM 和資料集。然後套用 LoRA，在這個資料集上有效率地微調模型。最後，它會將微調後的模型儲存回 Google Cloud Storage，以便日後用於文字生成或問答等應用程式。

**Airflow：模型服務** - 使用 vllm 在 GKE 上提供微調模型，以進行推論。

**Airflow：意見回饋迴路** - 每 xx 時間 (每小時、每天、每週) 重新訓練模型。

這張圖表說明在 GKE 上執行時，Airflow 2 的運作方式。



## 14. 微調模型與使用 RAG 的比較

本程式碼研究室會微調 LLM，而不是使用檢索增強生成 (RAG)。

現在來比較這兩種方法：

**微調：**建立專屬模型：微調可讓 LLM 配合特定工作或資料集，獨立運作，不需依賴外部資料來源。

**簡化推論：**不必使用獨立的擷取系統和資料庫，就能更快且更便宜地取得回覆，尤其是在經常使用的情況下。

**RAG：**依賴外部知識：RAG 會針對每項要求從知識庫擷取相關資訊，確保存取最新且具體的資料。

**增加複雜度：**在 Kubernetes 叢集等正式環境中導入 RAG 時，通常需要多個微服務來處理及擷取資料，可能會增加延遲時間和運算成本。

**選擇微調的原因：**

雖然 RAG 適合用於本程式碼研究室的小型資料集，但我們選擇微調，以示範 Airflow 的典型用途。這樣我們就能專注於工作流程協調層面，不必深入瞭解如何為 RAG 設定額外的基礎架構和微服務。

**結論：**

微調和 RAG 都是很有價值的技術，各有優缺點。最佳選擇取決於專案的具體需求，例如資料大小和複雜度、效能需求和成本考量。

---

步驟 15 · 約 5 分鐘

## 15. DAG 工作 #1 - 在 Airflow 上建立第一個步驟：下載資料

這個 DAG 單元的概觀：我們在容器映像檔中代管的 Python 程式碼，會從 Kaggle 下載最新的 RottenTomatoes 資料集。

請勿將這段程式碼複製到 GCS 值區。我們會在最後一個步驟複製 [mlops-dag.py](#)，其中包含一個 Python 指令碼中的所有 DAG 單元步驟。

mlops-dag.py

```

import yaml

from os import path
from datetime import datetime

from airflow import DAG
from airflow.models import Variable
from airflow.operators.python import PythonOperator
from airflow.providers.cncf.kubernetes.operators.pod import KubernetesPodOperator

from kubernetes import client, config
from kubernetes.client import models
from kubernetes.client.rest import ApiException

GCP_PROJECT_ID = Variable.get("GCP_PROJECT_ID")
BUCKET_DATA_NAME = Variable.get("BUCKET_DATA_NAME")
HF_TOKEN = Variable.get("HF_TOKEN")
KAGGLE_USERNAME = Variable.get("KAGGLE_USERNAME")
KAGGLE_KEY = Variable.get("KAGGLE_KEY")
JOB_NAMESPACE = Variable.get("JOB_NAMESPACE", default_var="airflow")

with DAG(dag_id="mlops-dag",
        start_date=datetime(2024,11,1),
        schedule_interval="@daily",
        catchup=False) as dag:

    # Step 1: Fetch raw data to GCS Bucket
    dataset_download = KubernetesPodOperator(
        task_id="dataset_download_task",
        namespace=JOB_NAMESPACE,
        image="us-central1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-
repo/dataset-download:latest",
        name="dataset-download",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "KAGGLE_USERNAME":KAGGLE_USERNAME,
            "KAGGLE_KEY":KAGGLE_KEY,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME
        }
    )

    dataset_download

```

步驟 16 · 約 5 分鐘

## 16. DAG 工作 #2 - 在 Airflow 上建立第二個步驟：資料準備

在本 DAG 單元中，我們會將 GCS 中的 CSV 檔案 (rotten\_tomatoes\_movie\_reviews.csv) 載入 Pandas DataFrame。

接著，我們使用 DATASET\_LIMIT 限制處理的資料列數量，以進行測試並提高資源效率，最後將轉換後的資料轉換為 Hugging Face 資料集。

仔細觀察後，您會發現我們在模型中訓練 1000 個資料列，並將「DATASET\_LIMIT」設為「1000」，這是因為使用 Nvidia L4 GPU 訓練這些資料需要 20 分鐘。

請勿將這段程式碼複製到 GCS 值區。我們會在最後一個步驟複製 [mlops-dag.py](#)，其中包含一個 Python 指令碼中的所有步驟。

```

import yaml

from os import path
from datetime import datetime

from airflow import DAG
from airflow.models import Variable
from airflow.operators.python import PythonOperator
from airflow.providers.cncf.kubernetes.operators.pod import KubernetesPodOperator

from kubernetes import client, config
from kubernetes.client import models
from kubernetes.client.rest import ApiException

GCP_PROJECT_ID = Variable.get("GCP_PROJECT_ID")
BUCKET_DATA_NAME = Variable.get("BUCKET_DATA_NAME")
HF_TOKEN = Variable.get("HF_TOKEN")
KAGGLE_USERNAME = Variable.get("KAGGLE_USERNAME")
KAGGLE_KEY = Variable.get("KAGGLE_KEY")
JOB_NAMESPACE = Variable.get("JOB_NAMESPACE", default_var="airflow")

with DAG(dag_id="mlops-dag",
        start_date=datetime(2024,11,1),
        schedule_interval="@daily",
        catchup=False) as dag:

    # Step 1: Fetch raw data to GCS Bucket
    dataset_download = KubernetesPodOperator(
        task_id="dataset_download_task",
        namespace=JOB_NAMESPACE,
        image="us-centrall1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-repo/dataset-download:latest",
        name="dataset-download",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "KAGGLE_USERNAME":KAGGLE_USERNAME,
            "KAGGLE_KEY":KAGGLE_KEY,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME
        }
    )

    # Step 2: Run GKEJob for data preparation
    data_preparation = KubernetesPodOperator(
        task_id="data_pipeline_task",
        namespace=JOB_NAMESPACE,
        image="us-centrall1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-repo/data-preparation:latest",
        name="data-preparation",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "GCP_PROJECT_ID":GCP_PROJECT_ID,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME,

```

```
        "DATASET_LIMIT": "1000",  
        "HF_TOKEN": HF_TOKEN  
    }  
)  
  
dataset_download >> data_preparation
```

---

步驟 17 · 約 5 分鐘

## 17. DAG 工作 #3 - 在 Airflow 上建立第三個步驟：微調模型

做為這個 DAG 單元的總覽，我們在這裡執行 `finetune.py`，使用新資料集微調 Gemma 模型。

請勿將這段程式碼複製到 GCS 值區。我們會在最後一個步驟複製 [mlops-dag.py](#)，其中包含一個 Python 指令碼中的所有步驟。

`mlops-dag.py`

```

import yaml

from os import path
from datetime import datetime

from airflow import DAG
from airflow.models import Variable
from airflow.operators.python import PythonOperator
from airflow.providers.cncf.kubernetes.operators.pod import KubernetesPodOperator

from kubernetes import client, config
from kubernetes.client import models
from kubernetes.client.rest import ApiException

GCP_PROJECT_ID = Variable.get("GCP_PROJECT_ID")
BUCKET_DATA_NAME = Variable.get("BUCKET_DATA_NAME")
HF_TOKEN = Variable.get("HF_TOKEN")
KAGGLE_USERNAME = Variable.get("KAGGLE_USERNAME")
KAGGLE_KEY = Variable.get("KAGGLE_KEY")
JOB_NAMESPACE = Variable.get("JOB_NAMESPACE", default_var="airflow")

with DAG(dag_id="mlops-dag",
        start_date=datetime(2024,11,1),
        schedule_interval="@daily",
        catchup=False) as dag:

    # DAG Task 1: Fetch raw data to GCS Bucket
    dataset_download = KubernetesPodOperator(
        task_id="dataset_download_task",
        namespace=JOB_NAMESPACE,
        image="us-centrall1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-repo/dataset-download:latest",
        name="dataset-download",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "KAGGLE_USERNAME":KAGGLE_USERNAME,
            "KAGGLE_KEY":KAGGLE_KEY,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME
        }
    )

    # DAG Task 2: Run GKEJob for data preparation
    data_preparation = KubernetesPodOperator(
        task_id="data_pipeline_task",
        namespace=JOB_NAMESPACE,
        image="us-centrall1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-repo/data-preparation:latest",
        name="data-preparation",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "GCP_PROJECT_ID":GCP_PROJECT_ID,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME,

```

```

        "DATASET_LIMIT": "1000",
        "HF_TOKEN": HF_TOKEN
    }
)

# DAG Task 3: Run GKEJob for fine tuning
fine_tuning = KubernetesPodOperator(
    task_id="fine_tuning_task",
    namespace=JOB_NAMESPACE,
    image="us-central1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-
repo/finetuning:latest",
    name="fine-tuning",
    service_account_name="airflow-mlops-sa",
    startup_timeout_seconds=600,
    container_resources=models.V1ResourceRequirements(
        requests={"nvidia.com/gpu": "1"},
        limits={"nvidia.com/gpu": "1"}
    ),
    env_vars={
        "BUCKET_DATA_NAME": BUCKET_DATA_NAME,
        "HF_TOKEN": HF_TOKEN
    }
)

dataset_download >> data_preparation >> fine_tuning

```

---

步驟 18 · 約 5 分鐘

## 18. DAG 工作 #4 - 在 Airflow 上建立最後一個步驟：推論 / 提供模型

[vLLM](#) 是強大的開放原始碼程式庫，專為 LLM 的高效能推論作業而設計。在 Google Kubernetes Engine (GKE) 上部署時，這項服務會運用 Kubernetes 的擴充性和效率，有效提供 LLM 服務。

步驟摘要：

- 將 DAG 「mlops-dag.py」上傳至 GCS bucket。
- 將兩個 Kubernetes YAML 設定檔複製到 GCS bucket，以設定推論。

mlops-dag.py

```

import yaml

from os import path
from datetime import datetime

from airflow import DAG
from airflow.models import Variable
from airflow.operators.python import PythonOperator
from airflow.providers.cncf.kubernetes.operators.pod import KubernetesPodOperator

from kubernetes import client, config
from kubernetes.client import models
from kubernetes.client.rest import ApiException

GCP_PROJECT_ID = Variable.get("GCP_PROJECT_ID")
BUCKET_DATA_NAME = Variable.get("BUCKET_DATA_NAME")
HF_TOKEN = Variable.get("HF_TOKEN")
KAGGLE_USERNAME = Variable.get("KAGGLE_USERNAME")
KAGGLE_KEY = Variable.get("KAGGLE_KEY")
JOB_NAMESPACE = Variable.get("JOB_NAMESPACE", default_var="airflow")

def model_serving():
    config.load_incluster_config()
    k8s_apps_v1 = client.AppsV1Api()
    k8s_core_v1 = client.CoreV1Api()

    while True:
        try:
            k8s_apps_v1.delete_namespaced_deployment(
                namespace="airflow",
                name="inference-deployment",
                body=client.V1DeleteOptions(
                    propagation_policy="Foreground", grace_period_seconds=5
                )
            )
        except ApiException:
            break
    print("Deployment inference-deployment deleted")

    with open(path.join(path.dirname(__file__), "inference.yaml")) as f:
        dep = yaml.safe_load(f)
        resp = k8s_apps_v1.create_namespaced_deployment(
            body=dep, namespace="airflow")
        print(f"Deployment created. Status='{resp.metadata.name}'")

    while True:
        try:
            k8s_core_v1.delete_namespaced_service(
                namespace="airflow",
                name="llm-service",
                body=client.V1DeleteOptions(
                    propagation_policy="Foreground", grace_period_seconds=5
                )
            )

```

```

        )
    )
    except ApiException:
        break
print("Service llm-service deleted")

with open(path.join(path.dirname(__file__), "inference-service.yaml")) as f:
    dep = yaml.safe_load(f)
    resp = k8s_core_v1.create_namespaced_service(
        body=dep, namespace="airflow")
    print(f"Service created. Status='{resp.metadata.name}''")

with DAG(dag_id="mlops-dag",
        start_date=datetime(2024,11,1),
        schedule_interval="@daily",
        catchup=False) as dag:

    # DAG Step 1: Fetch raw data to GCS Bucket
    dataset_download = KubernetesPodOperator(
        task_id="dataset_download_task",
        namespace=JOB_NAMESPACE,
        image="us-central1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-
repo/dataset-download:latest",
        name="dataset-download",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "KAGGLE_USERNAME":KAGGLE_USERNAME,
            "KAGGLE_KEY":KAGGLE_KEY,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME
        }
    )

    # DAG Step 2: Run GKEJob for data preparation
    data_preparation = KubernetesPodOperator(
        task_id="data_pipeline_task",
        namespace=JOB_NAMESPACE,
        image="us-central1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-
repo/data-preparation:latest",
        name="data-preparation",
        service_account_name="airflow-mlops-sa",
        env_vars={
            "GCP_PROJECT_ID":GCP_PROJECT_ID,
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME,
            "DATASET_LIMIT": "1000",
            "HF_TOKEN":HF_TOKEN
        }
    )

    # DAG Step 3: Run GKEJob for fine tuning
    fine_tuning = KubernetesPodOperator(
        task_id="fine_tuning_task",
        namespace=JOB_NAMESPACE,

```

```

        image="us-central1-docker.pkg.dev/{{ var.value.GCP_PROJECT_ID }}/mlops-airflow-
repo/finetuning:latest",
        name="fine-tuning",
        service_account_name="airflow-mlops-sa",
        startup_timeout_seconds=600,
        container_resources=models.V1ResourceRequirements(
            requests={"nvidia.com/gpu": "1"},
            limits={"nvidia.com/gpu": "1"}
        ),
        env_vars={
            "BUCKET_DATA_NAME":BUCKET_DATA_NAME,
            "HF_TOKEN":HF_TOKEN
        }
    )

# DAG Step 4: Run GKE Deployment for model serving
model_serving = PythonOperator(
    task_id="model_serving",
    python_callable=model_serving
)

dataset_download >> data_preparation >> fine_tuning >> model_serving

```

將 Python 指令碼 (DAG 檔案) 和 Kubernetes 資訊清單上傳至 DAGS GCS bucket。

```

gcloud storage cp mlops-dag.py gs://${BUCKET_DAGS_NAME}
gcloud storage cp manifests/inference.yaml gs://${BUCKET_DAGS_NAME}
gcloud storage cp manifests/inference-service.yaml gs://${BUCKET_DAGS_NAME}

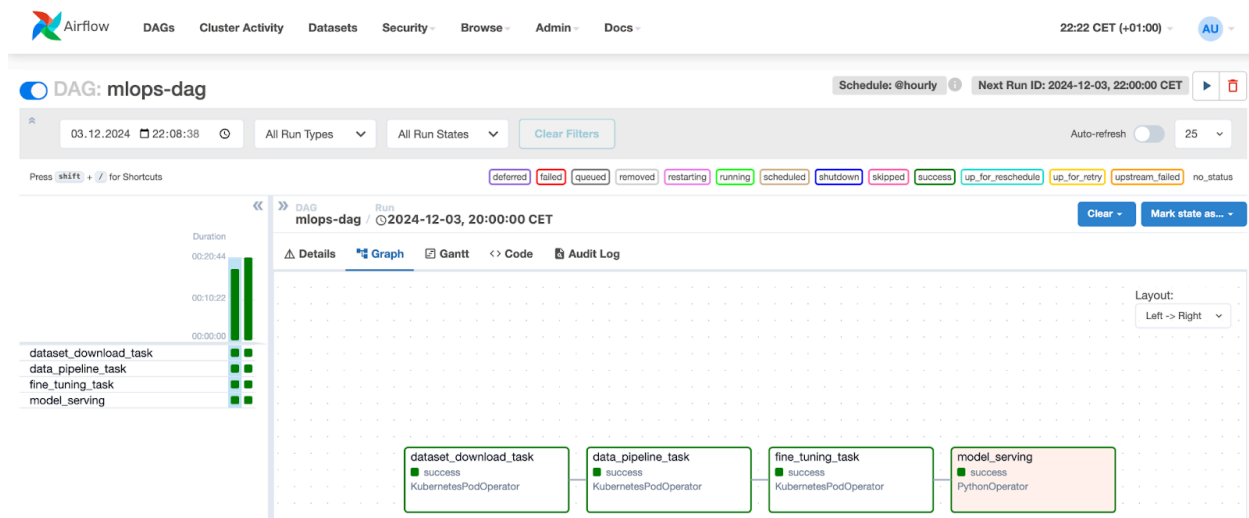
```

在 Airflow 使用者介面中，您會看到 mlops-dag。

1. 選取「取消暫停」。
2. 選取「觸發 DAG」，手動執行 MLOps 週期。

The screenshot shows the Airflow web interface. At the top, there's a navigation bar with links for DAGs, Cluster Activity, Datasets, Security, Browse, Admin, and Docs. The 'DAGs' tab is selected. Below the navigation bar, there's a section titled 'DAGs'. It includes filters for 'All' (1), 'Active' (0), 'Paused' (1), 'Running' (0), and 'Failed' (0). There's also a search bar and an 'Auto-refresh' button. Below these filters is a table of DAGs. The table has columns for DAG, Owner, Runs, Schedule, Last Run, Next Run, Recent Tasks, Actions, and Links. The first row shows the 'mlops-dag' DAG, owned by 'airflow', with a schedule of '@hourly'. The 'Last Run' column shows '2024-11-17, 18:00:00'. The 'Recent Tasks' column shows a series of circles representing task status. The 'Actions' column has buttons for 'Run', 'Pause', and 'Clear'. The 'Links' column has a three-dot menu icon. At the bottom right, it says 'Showing 1-1 of 1 DAGs'.

DAG 完成後，Airflow UI 會顯示類似下方的輸出內容。



完成最後一個步驟後，您就可以取得模型端點，並傳送提示來測試模型。

請先等待約 5 分鐘，再發出 curl 指令，以便開始推斷模型，並讓負載平衡器指派外部 IP 位址。

```
export MODEL_ENDPOINT=$(kubectl -n airflow get svc/llm-service --output  
jsonpath='{.status.loadBalancer.ingress[0].ip}')  
  
curl -X POST http://${MODEL_ENDPOINT}:8000/generate -H "Content-Type: application/json" -d @-  
<<EOF  
{  
  "prompt": "Question: Review analysis for movie 'dangerous_men_2015'",  
  "temperature": 0.1,  
  "top_p": 1.0,  
  "max_tokens": 128  
}  
EOF
```

輸出內容：

```
{"text":["Question: Review analysis for movie 'dangerous_men_2015'\nContext: The film is a mess, but it's a mess  
that's occasionally entertaining.\nAnswer: POSITIVE\nReason: While the film is a mess, it's a mess that's  
occasionally entertaining.\n"]}
```

步驟 19 · 約 0 分鐘

## 19. 恭喜！

您已使用 DAG 管道，在 GKE 上的 Airflow 2 建立第一個 AI 工作流程。

別忘了取消佈建已部署的資源。

---

## 20. 在正式環境中執行這項操作

雖然本程式碼研究室已提供絕佳的洞察資訊，協助您在 GKE 上設定 Airflow 2，但在實際情況中，您會在正式環境中執行這項操作時，考量下列主題。

使用 [Gradio](#) 或類似工具實作網頁前端。

您可以[在這裡](#)為工作負載設定自動應用程式監控，也可以[在這裡](#)匯出 Airflow 的指標。

您可能需要較大的 GPU，才能更快微調模型，尤其是使用較大的資料集時。不過，如果想在多個 GPU 上訓練模型，就必須分割資料集並分片訓練。以下說明 [PyTorch](#) 的 [FSDP](#) (完全分片資料平行，使用 [GPU 共用](#)達成此目標)。如要進一步瞭解，請參閱 [Meta](#) 的網誌文章，以及這篇 [Pytorch FSDP 教學課程](#)。

[Google Cloud Composer](#) 是代管式 Airflow 服務，因此您不必維護 Airflow 本身，只要部署 DAG 即可。

### 瞭解詳情

- Airflow 說明文件：<https://airflow.apache.org/>

### 授權

這項內容採用的授權為 Creative Commons 姓名標示 2.0 通用授權。

---